# Cheat Cheat Python – Programacion Avanzada

| | | |
|---|---|---|
| **Tipos da datos Básicos**<br>a = 8 #Entero<br>b = 8.0 #Flotante<br>c = 'abc' #Cadena de caracteres<br>d = 1.0+4j #Complejo<br>e = True #booleano puede ser False | **Operaciones aritméticas +**<br>1+1 => 2<br>1.0+2.0 => 3.0<br>'abc'+'def' => 'abcdef'<br>[1,2] + [2,1] => [1,2,2,1] | **Operaciones aritméticas ***<br>3*2 => 6<br>2.0*1.0 => 2.0<br>'abc'*3 => 'abcabcabc'<br>[1,2]*2 => [1,2,1,2] |
| **Operaciones aritméticas – y ****<br>2.0-1.0 => 1.0 #resta<br>2**3 => 8 #Potencia | **Operaciones aritméticas / y //**<br>1 / 2 => 0.5<br>3 // 2 => 1 (división entera) | **Operación % (residuo de la div)**<br>3%2 => 1<br>8%3 => 2 |
| **Operadores de pertenencia**<br>'a' **in** 'abc' => True<br>'a' **not in** 'abc' => False | **Operadores de identidad**<br>X = True<br>X **is** True => True<br>X **is not** True => False | **Operadores lógicos**<br>True **and** False => False<br>True **or** False => True<br>Not True => False |
| **Operadores relacionales => Bool**<br>**==** #Igualdad<br>**!=** #Diferente<br>**>** #Mayor que<br>**>=** #Mayor o igual<br>**<** #Menor que<br>**<=** #Menor o igual | **Estructura condicional**<br>**If** condición:<br>#bloque de código<br>**elif** condición2:<br>#bloque de código<br>**else**:<br>#bloque de código | **Combinacion de condiciones**<br>**If** (condición1 and condicion2):<br><br>**If** (condicion1 or condicion2):<br><br>**If** not (condicion1): |
| **Ciclo mientras**<br>**while** (condición):<br>#bloque de código | **Ciclo para**<br>**for** element **in** range(1,11)**:** #[1,10]<br>print(element) #1,2,…,10 | **Objeto Iterable range**<br>**range**(start,stop+1,step)<br>**range**(10,-11,-1) => {10,9,...,-9,-10} |
| **Declaración break**<br>Rompe la ejecución de un ciclo<br>for k in range(2,10):<br>if k==5:<br>break #rompre el for | **Inline if**<br>#código **if (**condición**) else** #código | **List comprehension**<br>**lista = [**#código **for** element **in X]** |
| **Asignaciones especiales**<br>x += 1 # x = x + 1<br>x -= 2 # x = x – 2<br>x *= 3 # x = x * 3<br>x /= 2 # x = x / 2<br>x %= 2 # x = x % 2<br>x **= 2 # x = x ** 2<br>x //= 3 # x = x // 3 | **Operaciones con bits**<br>a = 2 (binario 10) b = 3 (binario 11)<br>a & b => 2 #binario 10 AND<br>a \| b => 3 #binario 11 OR<br>a ^ b => 1 #binario 01 XOR<br>~ a => -3 #binario 11111101 NOT<br>a >> b => 0 #desplaza. derecha<br>a << b => 16 #desplaza. izquierda | **Operaciones básicas sobres listas**<br>.append() .extend([])<br>.insert() .count()<br>.clear() .index()<br>.remove() .pop()<br>.reverse() .sort()<br>.copy() |
| **Cadena de caracteres** (son objetos iterables)<br>tex = 'Hola Mundo!'<br>for letra in tex:<br>print(letra) # imprime cada letra<br># en lineas separadas | **Cadena de caracteres: métodos**<br>alfa, num = 'A','1'<br>alfa.isalpha() => True es alfabetico<br>num.isalpha() => False no es alfab.<br>alfa.isnumeric() => False<br>num.isnumeric() => True<br>alfa.lower() => 'a' minuscula | **Propiedades print**<br>Imprime y salta una línea<br>print("Hola")<br><br>Imprime pero no salta una línea<br>print("Hola ", end="") |

La evaluación de las relaciones lógicas en condicionales se realiza en orden, por ejemplo:

(True and False and True), Para uniones AND, se evalúan las condiciones hasta que se encuentra un False, y la salida inmediatamente es False. Para uniones con OR, se evalúan de izquierda a derecha hasta que se encuentre un True.